The Chinese University of Hong Kong, Shenzhen
School of Data Science



DDA5001 Final Project — Large Language Models
Part I - Pre-training a Small GPT Model
Due: 23:59, Nov 27.

This project guides you the process of pre-training, post-training, and reasoning of large language models (LLMs).

Contents

1 Introduction **2**

2 Basics of Large Language Models (LLMs) **2**
2.1 Prompts . . . 2
2.2 Tokenization . . . 3
2.3 LLM Modeling and Training Formulation . . . 4
2.3.1 Language Modeling . . . 4
2.3.2 Learning Problem Formulation of LLMs . . . 4

3 Part I: Pre-train a GPT Model on Shakespeare Dataset **4**
3.1 Dataset . . . 5
3.2 Code structure . . . 5
3.3 Tasks . . . 6
3.3.1 Complete the `TODO` blocks. . . . 6
3.3.2 Compare optimizers with different learning rates . . . 6
3.4 Submission . . . 7

1 Introduction

Large Language Models (LLMs) are a transformative development in artificial intelligence and machine learning, with broad impact across applications such as text generation, translation, summarization, code synthesis, retrieval-augmented question answering tool use, and reasoning. Trained on large corpora of text (and increasingly multimodal data), these models can produce human-like outputs that are often coherent, informative, and contextually nuanced. Much of their capability arises from large-scale pre-training on next-token prediction using the Transformer architecture, hence the name "Generative Pre-trained Transformer" (GPT). During pre-training, models learn to predict the next token given previous context (see Section 2.3), implicitly acquiring linguistic structure, world knowledge, and useful reasoning patterns. Post-training is often utilized after pre-training to further improve LLMs' alignment with human needs and stimulate its downstream and reasoning capabilities.

One of the most well-known LLMs is perhaps `ChatGPT` (powered by OpenAI models such as GPT-3.5, GPT-4, GPT-4o, o1, o3, GPT-5, and GPT-6), developed by OpenAI, which has demonstrated remarkable capabilities in understanding and generating text. It is widely utilized. There are also other representative models such as Claude by Anthropic, Deepseek, Doubao by Seed, Gemini by Google, Llama by Meta, Qwen by Alibaba, to name a few. Nowadays, LLMs represent a significant advancement in the field of artificial intelligence, and many of us are using them every day.

In this project, we aim to study several important elements of LLMs, including a small-scaled pre-training, post-training, and reasoning.

2 Basics of Large Language Models (LLMs)

2.1 Prompts

A prompt is the model's steering wheel: a short specification that tells the model what to do, with what context, and in what style. Unlike a generic "input", a well-crafted prompt guides behavior, small phrasing changes can shift tone, reasoning depth, or accuracy.

Typical ingredients:

- Task: What is needed (question, math, summarize, translate, classify, plan).
- Context: The evidence to use (passage, table, code, query).
- Constraints/style: Format, length, tone, schema.

Common forms (brief examples):

- Instruction style:

  ```
  Summarize the paragraph below in 2 bullets, each <12 words.
  ```

- Few-shot (learn by example) style:

```
Q: 2+2?
A: 4
Q: 3+5?
A:
```

- Chain-of-thought (reason stepwise) style:

  ```
  Think step by step before giving the final answer.
  ```

- Structured output style:

  ```
  Return JSON: {"verdict": "...", "evidence": ["...", "..."]}.
  ```

- System + user (chat setup) style:

  ```
  [System] You are a precise, terse tutor.
  [User] Explain weight decay in one sentence.
  ```

Why it matters: Prompts guide correctness, reduce hallucinations, and make LLMs useful without retraining. Even a simple sentence is a prompt, e.g.,

```
Shenzhen is a great city. What do you think?
```

2.2 Tokenization

Before a model can process text, it must convert characters of the input prompt into discrete units called *tokens*. This conversion, *tokenization*, is handled by a *tokenizer*, which maps text to token IDs and back (via a vocabulary). Modern LLMs use subword tokenization (e.g., BPE, WordPiece, Unigram) to balance coverage and efficiency: Common words stay intact; rare words are split into meaningful pieces.

Why it matters:

- **Efficiency**: Fewer tokens per sentence means faster, cheaper inference.
- **Robustness**: Subwords handle misspellings, new words, and multilingual text.
- **Limits**: Context windows are measured in tokens, not characters.

Example (illustrative only; actual splits/IDs depend on the tokenizer):

```
Input: Shenzhen is a great city.
Subwords: Sh en zhen is a great city .
Token IDs: 50, 831, 46732, 318, 257, 1049, 1748, 13
```

Note: Different tokenizers (GPT-2 BPE, Llama SentencePiece, etc.) produce different splits and IDs.

For Part I, we use a character-level tokenizer with a vocabulary of 65 characters.

2.3 LLM Modeling and Training Formulation

2.3.1 Language Modeling

In almost all of the cases, LLM is an *auto-regressive* model (a.k.a. causal model), for predicting the next token. Given an input prompt $\boldsymbol{x}$, many language tasks are to predict what should be the next tokens $\boldsymbol{y}$, by choosing $\boldsymbol{y}$ that has the largest conditional probability

$$\mathbb{P}[\boldsymbol{y}|\boldsymbol{x}]. \tag{1}$$

This is exactly the same as how we perform classification in logistic regression.

In LLMs, it models the conditional probability in (1) using an *auto-regressive Transformer*. Mathematically, it has the model

$$\mathbb{P}_{\boldsymbol{W}}[\boldsymbol{y}|\boldsymbol{x}] = \prod_{j=1}^{m} \mathbb{P}_{\boldsymbol{W}}[y_j|\boldsymbol{x}, \boldsymbol{y}_{1:j-1}]. \tag{2}$$

Here, "$\mathbb{P}_{\boldsymbol{W}}$" is the transformer, and thus $\boldsymbol{W} \in \mathbb{R}^d$ encompasses all trainable parameters of the transformer, including the query, key, value, and output attention matrices, as well as the gate, up, and down projection matrices of each transformer layer. y_j is the j-th token in the output $\boldsymbol{y}$.

Hence, LLMs generate text in the following manner:

(1) Receiving your input prompt $\boldsymbol{x}$ (which may be a question or a story you would like the model to do the continuation) to the LLMs.

(2) The LLMs will sample a y_1 from its vocabulary according to the distribution $\mathbb{P}_{\boldsymbol{W}}[y_1|x]$.

(3) Append the newly generated token y_1 to the input, giving $[x, y_1]$. Repeat the aforementioned steps by inputting $[x, y_1]$ into the model, until the maximum length / other stopping criterion is reached, which gives the generation $\boldsymbol{y}$.

2.3.2 Learning Problem Formulation of LLMs

The learning problem of LLMs is simply formulated from the maximum likelihood estimation (MLE) principle. That is, given a set of training data pairs $\mathcal{D} = \{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}$, the MLE learning problem of LLMs can be formulated as minimizing the negative log-likelihood of the generation probability:

$$\widehat{\boldsymbol{W}} \leftarrow \underset{\boldsymbol{W}}{\operatorname{argmin}} \ \mathcal{L}(\boldsymbol{W}) = \sum_{i \in \mathcal{D}} \sum_{j=1}^{m} -\log\left(\mathbb{P}_{\boldsymbol{W}}[y_{i,j}|\boldsymbol{x}_i, \boldsymbol{y}_{i,1:j-1}]\right). \tag{3}$$

Both pre-training and finetuning are using this learning problem formulation. The major difference between them lies in that they use different dataset $\mathcal{D}$.

3 Part I: Pre-train a GPT Model on Shakespeare Dataset

Part I is a **group** project with at most two students per group. You will complete the core code for pre-training a small GPT model to generate Shakespeare-style text, then compare a few optimizer and learning-rate settings. We use a small dataset and a limited training budget to keep the experiments manageable.

3.1 Dataset

We use the Shakespeare dataset, with 1,003,854 character tokens in the training split and 111,540 in the validation split. The prepared data is in the repository's `p1/data/` directory. The following is a quick preview of the dataset:

```
First Citizen:
Before we proceed any further, hear me speak.

All:
Speak, speak.

First Citizen:
You are all resolved rather to die than to famish?

All:
Resolved. resolved.

First Citizen:
First, you know Caius Marcius is chief enemy to the people.

All:
We know't, we know't.
```

The text illustrates Shakespeare's dramatic writing style.

3.2 Code structure

Code and data: https://github.com/CUHKSZ-Course/DDA5001-26Fall.

Our code is based on nanoGPT[1]. The repository's `p1` folder contains the following main files:

```
README.md  # project description and quick start
train.py  # main training logic
model.py  # GPT model and optimizer calls
evaluate.py  # evaluation scores
plot.py  # loss curves and comparisons
sample.py  # text generation
export_results.py  # lightweight submission archive
config/  # experiment settings
data/  # prepared dataset and tokenizer
```

Run the commands below from the project folder containing `train.py`. To set up your own machine, follow `setup/README.md`. For classroom machines, see the separate *Workstation Quick Guide*.

[1] https://github.com/karpathy/nanoGPT

3.3 Tasks

3.3.1 Complete the TODO blocks.

Complete the seven guided TODO blocks in model.py, train.py, and plot.py. Use the supplied native torch.optim calls; optimizer internals and evaluation code are provided. Run the checks before and after completing the TODOs; unfinished TODOs are expected to fail:

```
python check_solution.py
```

What to complete. Follow the comments beside each todo(...) call:

1. causal_mask: allow attention to the current and previous positions only.
2. shifted_labels: shift sequence positions by one to form next-token targets.
3. optimizer_adamw: supply learning_rate to the given AdamW call.
4. optimizer_sgdm: supply sgdm_learning_rate to the given SGD call.
5. optimizer_muon: supply muon_learning_rate and muon_momentum to the given Muon call; preserve the auxiliary AdamW and parameter groups.
6. optimizer_step: call optimizer.step() once inside the supplied loop, after gradient accumulation and clipping.
7. plots: pass the prepared xs and ys to the plotting call.

Keep the supplied structure and other arguments unchanged. evaluate.py is complete: its loss is total negative log-likelihood divided by the number of prediction tokens, not a sequence-mean score.

We encourage you to use a coding agent to help you. If using an agent, follow AGENTS.md and the TODO comments. After completing or changing a TODO, rerun python check_solution.py. Fix failures without changing the tests. Once all checks pass, run the short CPU smoke test in the README; after it succeeds, start the six full experiments below.

3.3.2 Compare optimizers with different learning rates

Compare **SGDM, AdamW, and Muon**. For each optimizer, run its provided default learning rate and **one other learning rate you choose**: six separate runs in total. In a copy of config/train_shakespeare_char.py, select optimizer_name, set the corresponding learning rate, and choose a separate out_dir. Keep the remaining training settings fixed. Each run starts from random weights; the model preset does not load pretrained weights. The unchanged configuration runs one AdamW experiment:

```
python train.py config/train_shakespeare_char.py
```

Choose the other learning rate based on the loss curve: try a larger value if learning is slow and smooth, or a smaller value if the loss fluctuates strongly or rises. The training-token budget counts all prediction tokens processed, including repeated samples. Allow approximately three hours for the six runs on the tested hardware; runtime varies by device. If curves change their relative ordering (a crossover), discuss it briefly; no extra training is required to find one.

Each run saves `config.json`, `environment.json`, `metrics.csv`, `summary.json`, and `run.log` for your own plots or analyses. The codebase provides a training pipeline and basic plots. You may write your own analysis scripts or modify the plotting and analysis code.

Compare the final training-budget endpoint recorded in `last.pt`; `best.pt` may refer to an earlier checkpoint. For example, after the default AdamW run:

```
python plot.py --run-dir=runs/adamw-default
python sample.py --checkpoint=runs/adamw-default/last.pt
python plot.py --runs-dir=runs
```

These commands save loss plots and a text sample; generation starts from a newline by default.

3.4 Submission

1. A PDF report for Part I with **at most one page of main content**, plus at most two additional pages in total for references and an appendix. Include:

   - Show training and validation loss curves for the six settings, and report their endpoint validation losses and training times. Briefly analyze representative learning-rate effects and any crossover visible in these runs.
   - Display a short text sample from one endpoint checkpoint, identifying its optimizer and learning rate.
   - Describe the main difficulties and how you solved them. State each group member's contribution.

2. Submit one ZIP to the Part I assignment on Blackboard containing the report, completed Python code, and the six runs' configurations, environment information, CSV metrics, summaries, logs, plots, and representative generated text. You may package code and results with:

   ```
   python export_results.py --runs-dir=runs \
     --output=part1-results.zip
   ```

   Add your report PDF to the exported ZIP. **Do not upload checkpoints or the supplied dataset**; keep a separate backup of checkpoints for possible spot checks.

3. **Submit once per group. This Part I is no longer an individual work, unless you only have yourself in your group (i.e., individual forms a group).** If you have two members in your group, both members should contribute and be able to explain the completed code and reported observations.

There is no single target loss or generated passage, i.e., **no standard correct answer**; report and explain your own results. Your code must still pass the supplied checks.

Part I is worth 30 of the 100 points for the whole project.